ALGORITHM CHALLENGE 2026 · VERNEWELL GROUP

SphoorthiQ SAR Image Segmentation

Classical and quantum optimization on one verified objective: Markov Random Field, QUBO and QAOA, benchmarked on real MSTAR radar data

Company	SphoorthiQ Private Limited · https://sphoorthiq.com/
Challenge	Algorithm Challenge 2026, organized by Vernewell Group
Use-case	THALES Use-Case: SAR (Synthetic Aperture Radar) Image Segmentation
Submitted to	challenge@vernewellgroup.com
Subject / reference	Algorithm Challenge 2026
Submission deadline	10 November 2026
Team members	Dr. Kalakond Parvath, Dr. Santoshi, Soumyadip
Prepared by	Sangameshwar Swami, SphoorthiQ Private Limited
Date	November 2026

Final proposal submitted for evaluation by the jury and relevant stakeholders of the Algorithm Challenge 2026.

Contents

1. Executive summary 3

2. Problem and objectives 3

3. Proposed solution 4

4. The running application 5

5. Methodology 9

6. Results to date and expected outcomes 10

7. Supporting documentation and submission details 12

Appendix A. User interface and job execution details 13

Bibliographic references 15

List of figures

Figure 1. Solution architecture: one input, two solvers, one score 4

Figure 2. Segmentation result on a real MSTAR chip 6

Figure 3. Metrics and QAOA convergence 7

Figure 4. Pattern detection 7

Figure 5. Synthetic validation 8

Figure 6. Execution record on the quantum platform 9

Where to find what the organizers asked for

Proposed solution · section 3 4

Objectives · section 2 3

Methodology · section 5 9

Expected outcomes · section 6 10

Supporting documentation · section 7 12

Execution details: interface and quantum jobs · Appendix A 13

1. Executive summary

SphoorthiQ is a working application that segments real radar images of vehicles and benchmarks a quantum solver against an exact classical solver on the identical objective. Built for the THALES SAR Image Segmentation use-case, it formulates each image as a Markov Random Field, proves that formulation equivalent to a QUBO, and solves it with both graph cut and QAOA.

On two real chips the quantum solver matches the proven classical optimum at 9 and 16 qubits, and at every noise level tested at 9 qubits. It falls short at 25 qubits. At 9 qubits a random-sampling baseline with the same sample budget finds the optimum too, so that match shows the pipeline is correct, not that QAOA outperforms sampling. At 16 qubits random sampling rarely finds the optimum and at 25 qubits never does, and QAOA's energy is below the random average in all four of those rows. We report this as measured and do not claim a quantum advantage.

MSTAR has no per-pixel masks, so we scored the classical solvers against the SARBake overlay, whose masks are generated from 3D CAD models of the vehicles. On 120 held-out test chips the two-class target IoU is 0.56 and the three-class mean IoU is 0.66. On the small edge tiles the quantum solver can run on, the classical scores are much lower (target IoU 0.18 on 4×4 tiles), so those scores say more about the pipeline than about the quantum runs.

The proposal's value is a trustworthy benchmark: one verified problem, two solvers, and results that can be reproduced. All quantum results to date ran on Classiq simulators, not physical quantum hardware.

Key facts at a glance

2.13×10^{-14}
largest gap between the MRF and QUBO energies, checked over all 65,536 states of a 4×4 patch

100%
accuracy, IoU and Dice of the exact classical solver on the THALES worked example

52 of 52
automated tests passing, on real MSTAR radar chips with no mocks

0.56
target IoU of the classical solver on 120 held-out real chips, scored against the SARBake masks

51.8%
pattern-detection accuracy on 1,365 real test chips, against 33% by chance

~25 qubits
today's QAOA ceiling on the available simulators: a 5×5 tile for two labels

2. Problem and objectives

Synthetic aperture radar (SAR) images must be separated into target and background despite heavy speckle noise. The THALES brief asks for quantum computation to evaluate all pixel-neighbor interactions at once, for a fast and satisfactory segmentation, using a Markov Random Field formulation [20]. State-of-the-art approaches such as clustering and region-based methods [17][18][19] give sub-optimal results, and the brief itself notes that for an Ising prior the exact answer can be computed in polynomial time by graph cut. That sets a high bar, so the core question is how the quantum result compares with the best classical answer.

Our objectives are:

1. Formulate SAR segmentation as a Markov Random Field and convert it, verifiably, into a form quantum solvers accept.
2. Benchmark QAOA against an exact classical solver on the same objective and the same real data.
3. Measure where the quantum approach holds up and where it stops, by problem size and by noise level.

4. Cover the brief's requirements beyond binary segmentation: images without objects, low signal-to-noise images, edge detection, three or more classes, and pattern detection.
5. Deliver a running application rather than a standalone notebook, so reviewers can inspect each result.

3. Proposed solution

The solution is a single web application that takes one region of a real SAR image and solves it with two solvers side by side. Because both start from the same problem object and finish on the same energy objective, the comparison is direct rather than approximate.

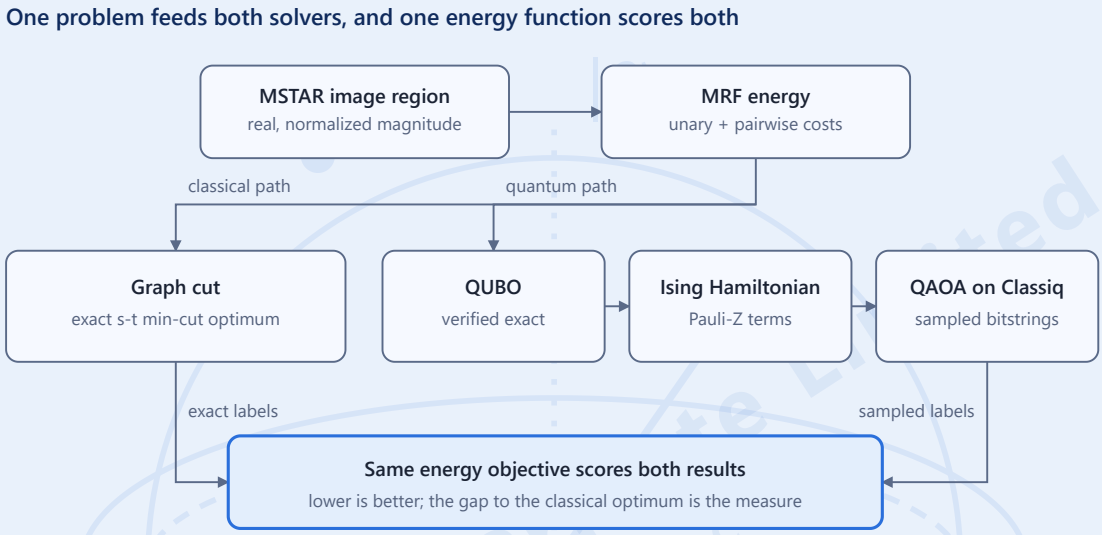


Figure 1. Solution architecture: one input, two solvers, one score.

The app runs as one FastAPI and React process on one port. It streams real per-iteration QAOA progress to the browser, and the classical path works without any quantum account.

Brief requirement	Status	What we did, and the caveat
Segment SAR images with an MRF, using quantum optimization	**Met**, on simulators	MRF, QUBO, then QAOA on Classiq. Every quantum run used a simulator; none used a physical quantum processor.
Classical baseline (the brief names a MATLAB genetic algorithm)	**Different baseline**	We benchmark against exact graph cut, the exact method the brief itself names for an Ising prior. It is a stronger reference than a heuristic. We did not run the genetic algorithm.
Validation on pre-segmented 64×64 MSTAR images	**Partly met**	The public MSTAR release provides 128×128 chips with no per-pixel masks, so the 64×64 size is not used. Real chips are scored against the SARBake overlay's CAD-derived masks: on 120 held-out chips the classical solver reaches a two-class target IoU of 0.56 and a three-class mean IoU of 0.66. Exact accuracy is also scored on synthetic cases, including the brief's own example.
Binary example (brief, page 3)	**Met** (classical)	Exact graph cut reproduces the printed label matrix. QAOA reaches 92% to 96% accuracy on this 25-qubit example.
Images without an object; low signal-to-noise images	**Met**	Verified target-free and low-contrast region presets on real chips.

Brief requirement	Status	What we did, and the caveat
Three or more classes (shadow as class 2)	**Partly met**	Background, target and shadow are implemented, with approximate alpha-expansion on the classical side. It does not reproduce the brief's printed three-class example: with a low background reference the solver agrees on 21 of 25 pixels (84%) but labels all four shadow pixels as background, and with a higher reference it agrees on 11 of 25 (44%). The application models a dark shadow, while the example's shadow has mid-level intensity. QAOA is limited to a 2×2 tile. Against the SARBake masks on real chips, the classical three-class result reaches a mean IoU of 0.66, with 0.72 for the shadow.
Edge detection, with the multi-class and single-pixel refinements	**Partly met**	The brief's written rule (any differing neighbor) is implemented, with a single-pixel option and an object-versus-shadow edge map. The brief's printed example marks only the object-side pixel of each boundary; our single-pixel option marks the top or left side, so it differs on that example.
Pattern detection (left undefined)	**Met**, by our interpretation	Target-type recognition across BMP2, BTR70 and T72: 51.79% on 1,365 real test chips, against 33% by chance.

4. The running application

The figures below are screenshots of the application running on real MSTAR data. They were captured in November 2026 from a local run of the application, and the quantum results come from live QAOA runs on Classiq's simulator.

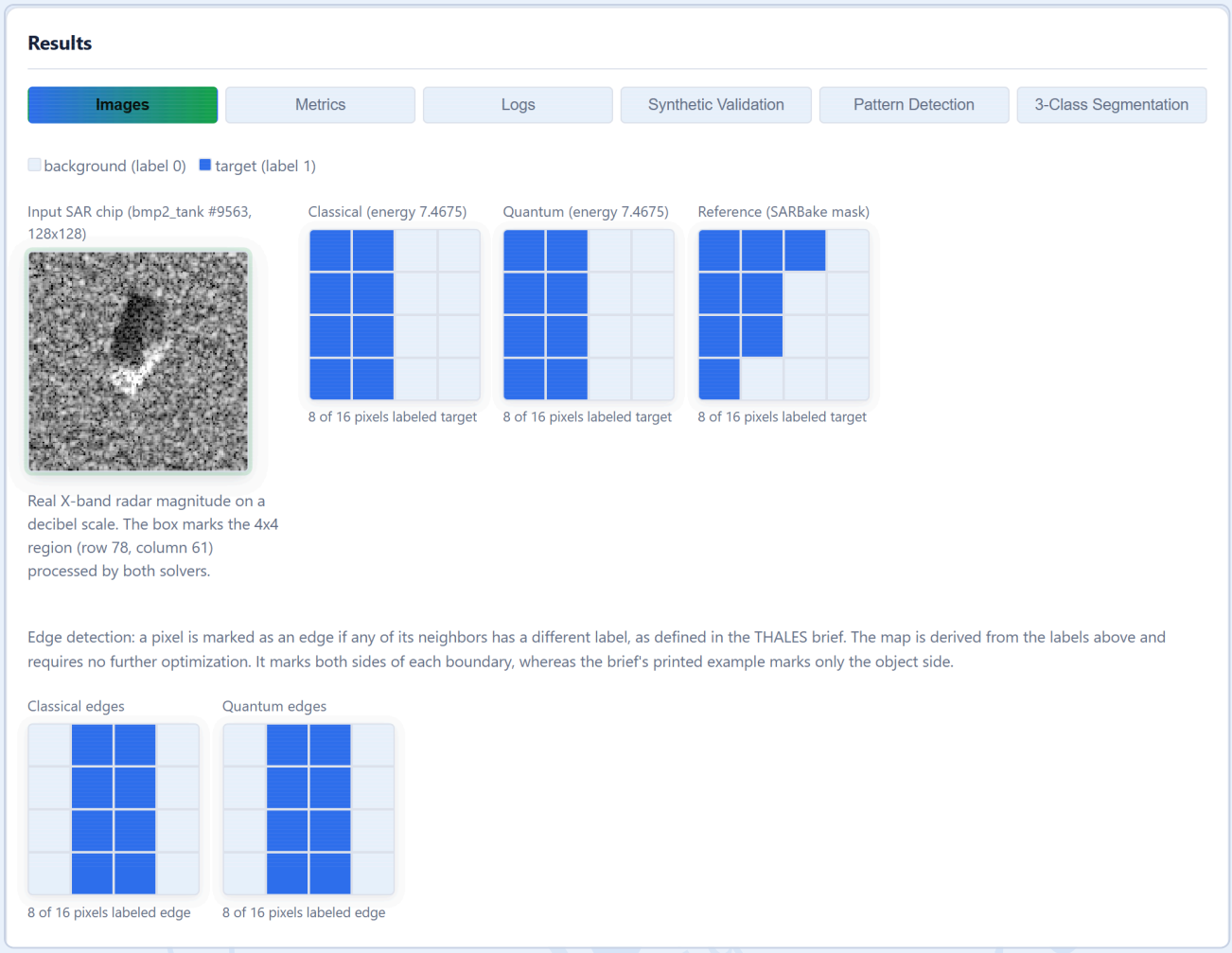


Figure 2. Segmentation result. Real MSTAR chip (BMP2 #9563), 4×4 region (16 qubits) on the vehicle's edge. The panel shows the input chip, the classical segmentation (energy 7.4675), the quantum segmentation (energy 7.4675), the SARBake reference mask, and the derived edge maps. On this run both solvers found the same labeling. QAOA is random, so a repeat run can return a different result, while the classical result is identical every time.

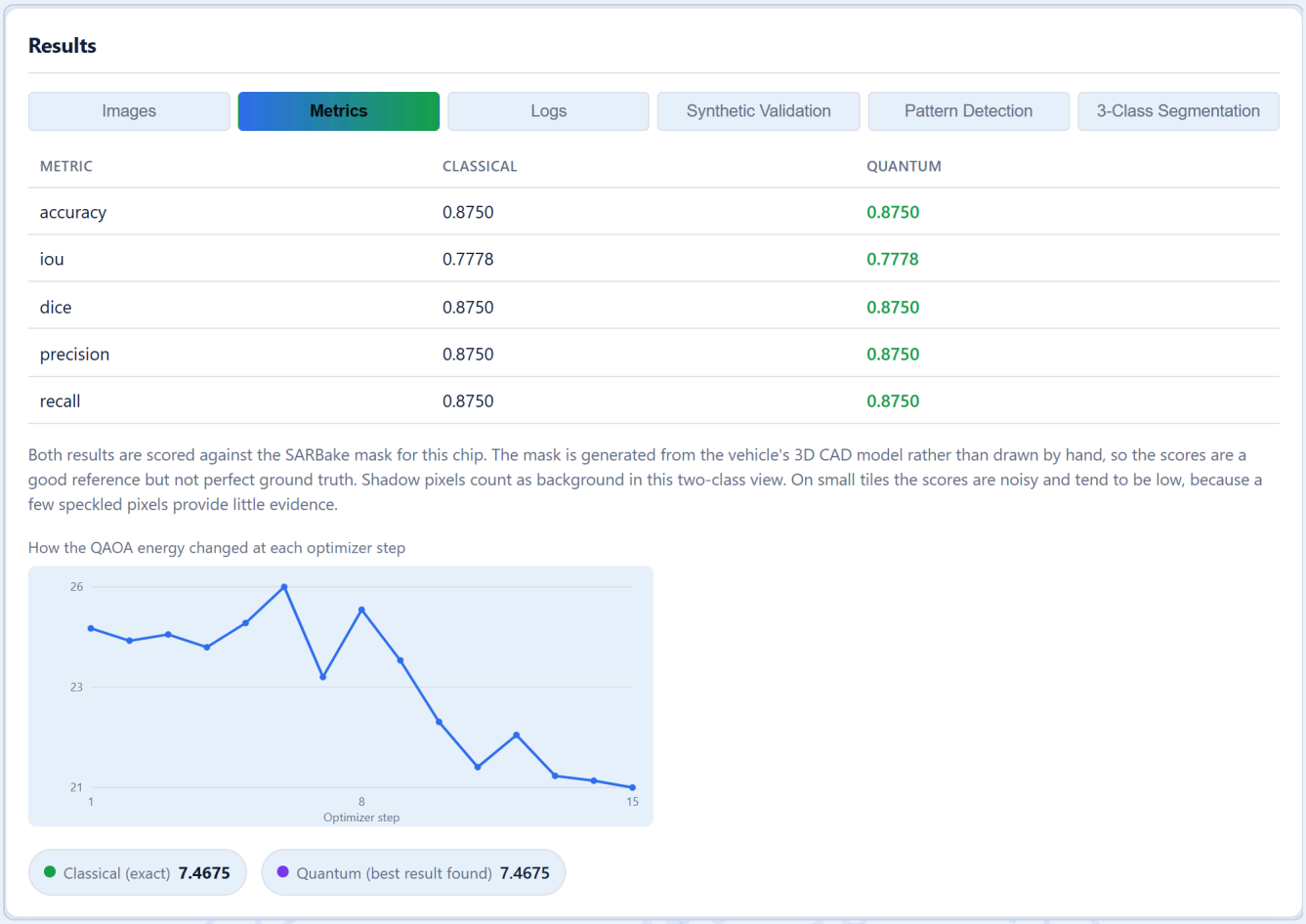


Figure 3. Metrics and QAOA convergence. Both results are scored against the SARBake mask (accuracy 0.875, IoU 0.78, Dice 0.88; the two solvers found the same labeling) and compared on energy. The chart plots the QAOA energy at each of the 15 optimizer steps, streamed live from Classiq; the classical exact value is shown beside the best quantum result found.

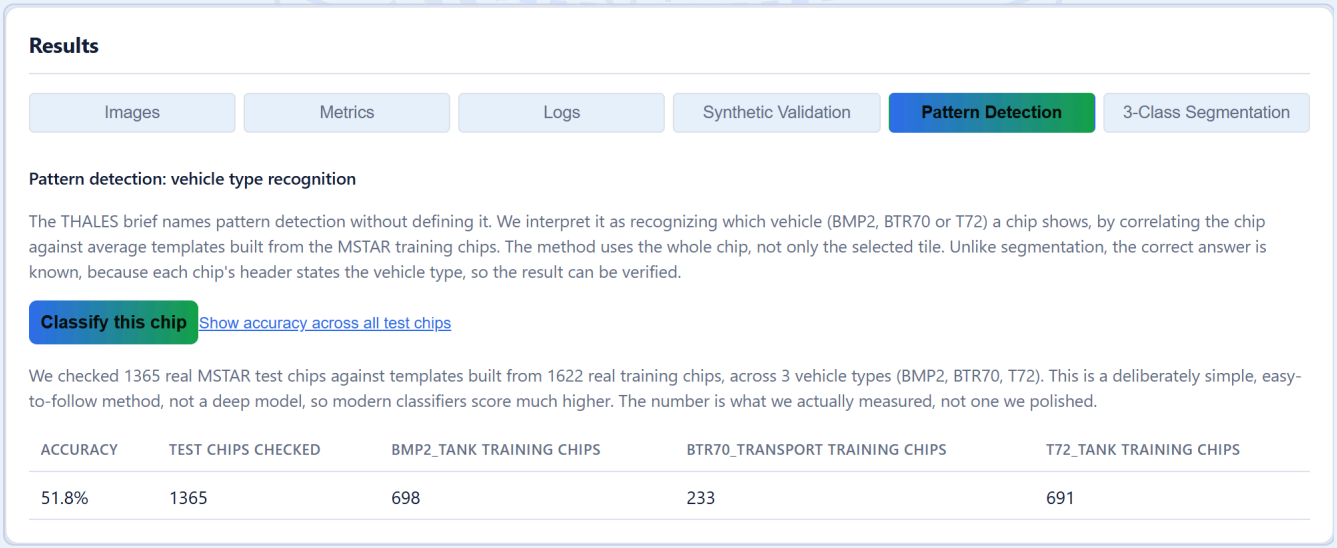


Figure 4. Pattern detection. Our interpretation of the brief's undefined pattern-detection requirement: recognizing which vehicle (BMP2, BTR70 or T72) a chip shows, checked against each chip's own header. The accuracy across all test chips is reported on this tab.



Figure 5. Synthetic validation. Accuracy, IoU and Dice on cases whose right answer is known by construction: the worked example quoted from the THALES brief and three generated cases. These are not MSTAR chips; real chips are scored against the SARBake masks (section 6). This is a fresh run with the quantum solver on: graph cut is exact on all four cases, while QAOA is exact on the three small cases and falls short on the 25-qubit THALES example (96.0% accuracy in this run).

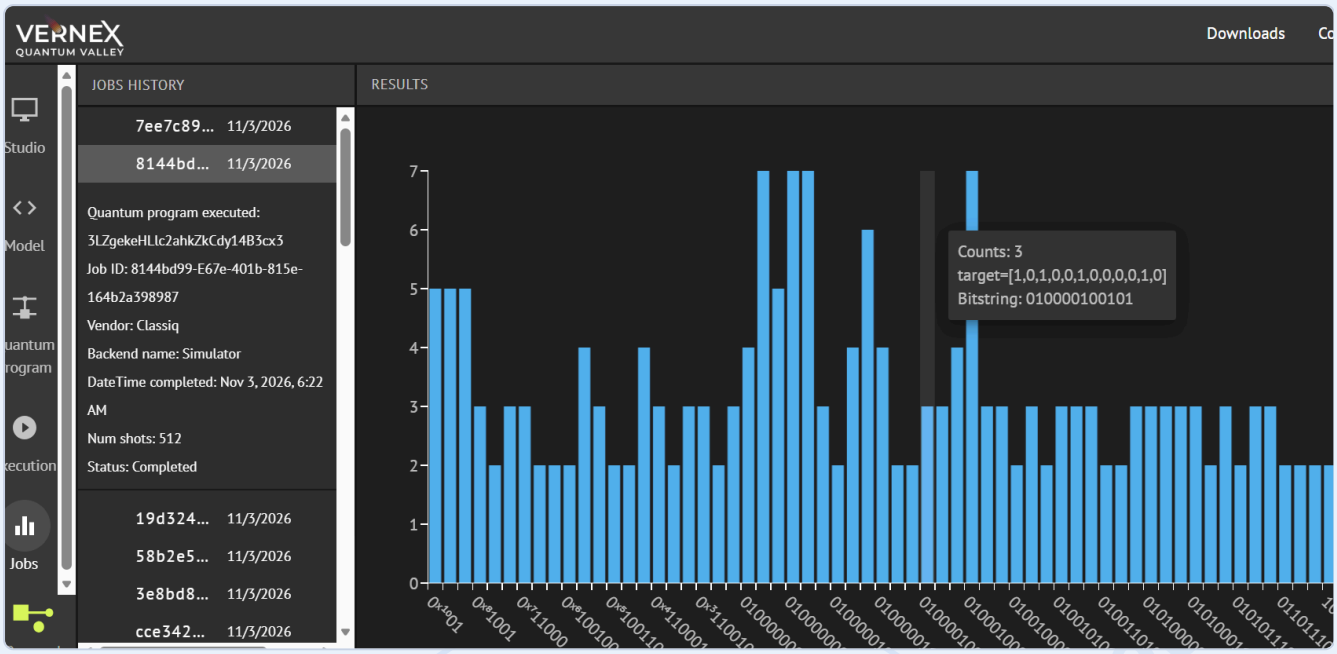


Figure 6. Execution record on the quantum platform. Job history of the cloud platform behind the Classiq SDK, showing a completed run from this project: job ID 8144bd99-e67e-401b-815e-164b2a398987, vendor Classiq, backend Simulator, 512 shots, status Completed, 3 November 2026. The chart is the distribution of measured bitstrings for that job; the tooltip shows one 12-bit outcome (counts: 3). This job is the application's three-class run on BMP2 #9563 (2×2 tile, 12 qubits) recorded at 06:21:36 to 06:22:43 (Appendix A.4), and it confirms that the quantum results were produced by Classiq's simulator, not by physical quantum hardware.

5. Methodology

We minimize one Markov Random Field energy with both solvers and verify each link between them. The energy has a unary term for how well a label fits a pixel's magnitude, and a pairwise term that penalizes disagreeing neighbors [1][3][20]:

$$E(x) = \Sigma_{i \in V} \theta_i(x_i, z_i) + \Sigma_{(i,j) \in E} \Psi_{i,j}(x_i, x_j)$$

The pairwise cost is reduced across real intensity edges, so object boundaries are not smoothed away.

1. **Data.** We use the MSTAR Public Targets chips [15]: X-band SAR images of T72, BMP2 and BTR70 vehicles at 128×128 pixels, plus a SLICY calibration target at 54×54. Pattern detection trains on the 17° depression-angle split and tests on the 15° split, the standard protocol [10][11]. The brief describes pre-segmented 64×64 MSTAR images; the public release provides 128×128 chips with no per-pixel masks. For masks we use the SARBake overlay [21], which labels every T72, BMP2 and BTR70 chip as background, target or shadow from a 3D CAD model of the vehicle at the chip's viewing angle. It is a good reference, not hand-drawn ground truth: it covers the bright vehicle return rather than every pixel a person might outline. SLICY has no masks.
2. **Unary term.** Each pixel's cost comes from its brightness in decibels against its own chip's background, as a robust z-score (median and median absolute deviation of the decibel image). A few constants decide where a pixel starts to count as target and how bright target and shadow are expected to be. We set them on the MSTAR TRAIN chips (17°) and score on the TEST chips (15°), so the reported scores are on chips the constants never saw. There is no trained classifier. An earlier version used the log-scaled, min-max normalized magnitude directly as the target probability; scored against the masks it marked about 13 of roughly 320 target pixels, which is why it was replaced.

3. **Classical solver.** An s-t min-cut graph cut gives the exact optimum for two labels, with no qubit limit [1][2]. It is valid because the pairwise term is a Potts penalty, which is always submodular [4]. The brief names a MATLAB genetic algorithm as its baseline; graph cut is a stronger reference, and we did not run the genetic algorithm.
4. **Quantum solver.** The energy becomes a QUBO, then an Ising Hamiltonian [5], then a QAOA circuit [6] built from Classiq's `qaoa_init` and `qaoa_layer` functions [14]. A COBYLA optimizer [8] tunes the circuit for 15 iterations by default, from a linear-ramp starting point (cf. [7]). Every sampled bitstring is scored on the real QUBO energy, so the quantum result is judged on the same number as the classical one. A final 2,048-shot sample at the tuned parameters picks the best bitstring (Appendix A.2).
5. **Three-class extension.** Background, target and shadow use a one-hot QUBO with a penalty enforcing one label per pixel. Alpha-expansion [3] is the classical baseline, and it is approximate. With 3 qubits per pixel, the 25-qubit ceiling allows a 2×2 tile (12 qubits); a 3×3 tile would need 27.
6. **Pattern detection.** A correlation score compares each chip's target patch with per-class templates averaged from the training split. This is a simple relative of correlation-filter recognition [12], not a reimplementation of it; target-variant recognition on MSTAR is also studied in [13].

Evaluation uses two kinds of reference. Real chips are scored against the SARBake masks, which are CAD-derived and so a good guide rather than perfect truth, and quantum runs are also compared with the classical optimum on energy. Synthetic cases with known answers, including the worked example in the THALES brief, give exact accuracy, IoU and Dice.

We chose QAOA over quantum annealing, the route used in [9], because Classiq is a gate-model platform. We chose one-hot encoding despite its cost of 3 qubits per pixel, because its validity constraint is simple to verify.

The QUBO step is checked exhaustively: over all 65,536 states of a 4×4 patch, the largest gap between the MRF energy and the QUBO energy is 2.13×10^{-14}, and graph cut matches the brute-force optimum.

6. Results to date and expected outcomes

Graph cut is exact at every size tested. QAOA reaches the classical optimum at 9 and 16 qubits on both chips and falls short at 25 qubits. The table compares the lowest MRF energy found (lower is better) on two real MSTAR test chips, using the default QAOA budget of 15 optimizer iterations. Each tile sits on the vehicle's edge, where about half of it is target according to the SARBake mask, so the solver has a real decision to make. The random-sampling column is a baseline we added: the best of about 9,700 uniformly random labelings, matching QAOA's sample budget, averaged over 30 seeds. The last column scores each result against the mask.

Chip	Qubits	Classical optimum	QAOA	Random sampling, best (mean of 30)	IoU, classical / QAOA
T72 #132	9	6.5307	6.5307	6.5307	0.40 / 0.40
BMP2 #9563	9	5.3805	5.3805	5.3805	0.75 / 0.75
T72 #132	16	11.3321	11.3321	12.1226	0.44 / 0.44
BMP2 #9563	16	7.4675	7.4675	8.7559	0.78 / 0.78
T72 #132	25	13.8564	14.3462	20.1669	0.59 / 0.57
BMP2 #9563	25	12.1758	13.2295	18.4777	0.71 / 0.67

The reading is cautious. At 9 qubits there are only 512 labelings, so random sampling finds the optimum on every seed and the match proves little. At 16 qubits random sampling found the optimum on only 6 and 3 of 30 seeds, and at 25 qubits on none, and QAOA's energy was below the random average in all four of those rows. Each QAOA value is a single run, and QAOA is random, so repeat runs can differ. We do not claim a quantum advantage: the classical solver is exact and takes milliseconds. Graph cut also solved 8×8 edge patches (four times QAOA's qubit ceiling) in about 3 ms and 2 ms, with energies of 34.1487 (T72 #132) and 27.4969 (BMP2 #9563) and IoU of 0.71 and 0.78 against the mask.

Real-mask evaluation

We scored the classical solvers on 120 random MSTAR TEST chips (40 each of BMP2, BTR70 and T72, 15° depression), using a 48×48 region around each vehicle. The decibel setup was tuned on the TRAIN chips only. No quantum jobs were run for this table. The masks are CAD-derived, so the scores are a good guide rather than perfect truth.

Score on 120 TEST chips	Earlier setup	Decibel setup
Two classes: target IoU	0.04	0.56
Two classes: target Dice	0.07	0.70
Two classes: precision / recall	0.99 / 0.04	0.79 / 0.66
Two classes: chips with IoU above 0.5	0 of 120	89 of 120
Three classes: pixel accuracy	0.44	0.82
Three classes: mean IoU	0.16	0.66
Three classes: target / shadow IoU	0.03 / 0.00	0.61 / 0.72

The earlier setup marked about 13 target pixels where the mask has about 320, so nearly everything came out as background. Per vehicle, two-class IoU with the decibel setup is 0.55 for BMP2, 0.50 for BTR70 and 0.62 for T72.

On the small edge tiles the quantum solver can run on, scores are much lower. Each tile is placed where the mask is about half target, the hardest case. Averaged over the same 120 chips, target IoU is 0.12 on 2×2 tiles, 0.18 on 4×4 and 0.34 on 8×8, and three-class pixel accuracy is 0.32 to 0.36, below the 0.50 from always choosing the most common class. The two chips in the scaling table score higher than these averages, so their IoU is not typical.

Other measured results:

- **Noise robustness.** On a 9-qubit edge tile, QAOA matched the classical optimum at all six noise levels from σ = 0 to 0.3, subject to the same caveat about random sampling at that size. Energies cannot be compared between noise levels, because each is a different image. Scores against the mask were 0.40 with no added noise, 0.56 to 0.62 at moderate noise and 0.50 at σ = 0.3, the same for both solvers because they found the same labeling. On a 9-pixel tile one pixel moves IoU a lot, so we read no trend into it.
- **Synthetic validation.** Graph cut reproduces the THALES worked example exactly (accuracy, IoU and Dice all 1.00). QAOA is stochastic and fell short of the classical solver on that 25-qubit example in all four runs we made: accuracy 0.96 (IoU 0.92, Dice 0.96) in two runs and 0.92 (IoU 0.85, Dice 0.92) in the other two. Figure 5 shows the fourth run, at 0.96. It scored 1.00 on the smaller synthetic cases in every run.
- **The brief's other printed examples.** On the three-class example, alpha-expansion agrees with the printed labels on 21 of 25 pixels (84%) when its background reference is 0.2 or lower, but labels all four shadow pixels as background; with a reference of 0.3 or higher it agrees on 11 of 25 (44%). The application models

the shadow as dark, while the example's shadow has mid-level intensity. On the edge example, the printed matrix matches marking only the object-side pixel of each boundary; the application's both-sides map differs from it on 5 pixels and its top-or-left single-pixel map on 1.

- **Pattern detection.** 51.79% accuracy on 1,365 real test chips against 33% chance. This is a transparent baseline, well below the roughly 95% reported for CNN-based MSTAR classifiers.
- **Software checks.** All 52 automated tests pass. They run on real MSTAR chips, with no mocks.

Expected outcomes

1. A reproducible benchmark in which two solvers are scored on one verified objective, with repository scripts that regenerate the scaling, robustness and validation tables.
2. An assessment of where quantum optimization stands today: correct but not yet competitive, capped at about 25 qubits on the Classiq simulators.
3. A base for the next stages in our roadmap: tiling a full 128×128 chip into overlapping regions, running on physical quantum hardware, and extending beyond three labels.

7. Supporting documentation and submission details

This proposal is submitted to challenge@vernewellgroup.com with the subject line "Algorithm Challenge 2026", by 10 November 2026.

Supporting material in the repository:

- **Algorithm Design Dossier** (`docs/Algorithm_Design_Dossier.md`): problem framing, objective, constraints, assumptions, candidate formulations, scalability, robustness and limitations, section by section.
- **README**: setup, run and deploy steps, the API table, environment variables and known limitations.
- **Scripts**: `scaling_sweep.py`, `robustness_sweep.py` and `synthetic_validation_run.py` regenerate the result tables against a running instance of the app. `sarbake_eval.py` reproduces the real-mask scores and `random_baseline.py` the random-sampling column; neither needs a Classiq session.
- **Tests**: 52 pytest tests on real MSTAR chips, including the exhaustive QUBO equivalence check.
- **Evidence file**: `datasets/reports/11_mrf_qubo_verification.json`.
- **Execution record**: the quantum platform's job history for runs of this application (Figure 6), and the application's own job log (Appendix A).
- **Running application**: a FastAPI backend and React interface in one process, with live QAOA progress.

Known limits:

- QAOA is capped near 25 qubits, which allows a 5×5 tile for two labels and a 2×2 tile for three.
- All quantum runs used Classiq simulators; no physical quantum processor has been used.
- Real-chip scores use the SARBake masks, which are generated from CAD models, so they are a good guide rather than hand-drawn truth. Chips without a mask (SLICY) are compared on energy only.
- On small edge tiles the scores are much lower (section 6), so the real-mask scores for the quantum runs are weak evidence. The decibel constants were tuned on a small grid. The real-mask scores are for the classical solvers; QAOA was scored only on edge tiles of up to 5×5 pixels.
- Three-class graph cut is approximate, not exact.
- The brief's genetic-algorithm baseline was not run; the classical reference is exact graph cut.
- The brief's printed three-class and edge examples are not reproduced exactly (section 3).
- The brief describes 64×64 pre-segmented images; we use 128×128 public chips, with masks from SARBake.

Appendix A. User interface and job execution details

This appendix documents how the application is used through its web interface, what happens when a quantum job runs, and the jobs the application has recorded. All figures come from the application's own job history (`datasets/reports/jobs`), read in November 2026. Times are India Standard Time (UTC+5:30).

A.1 Using the web interface

Users sign in, then work in one page with an input panel on the left and result tabs on the right. Every call to the backend carries the signed-in session token.

View	What the user does	Backend call
Images	Chooses a real MSTAR chip and a region (row, column, tile size) or a preset (Target, No object, Low SNR), optionally ticks "Also run the quantum solver", and clicks Run pipeline. Sees the chip, both segmentations, the SARBake reference mask and the edge maps.	`POST /api/mstar` or `/api/mstar/stream` (live QAOA progress)
Metrics	Reads the classical and quantum energies, their scores against the reference mask, and the QAOA convergence chart.	Same response as the run
Logs	Reads the run log for the current result.	Same response as the run
Synthetic Validation	Runs the THALES worked example and three generated cases; optionally with the quantum solver.	`POST /api/mstar/synthetic-validation`
Pattern Detection	Classifies one chip, or runs the accuracy check across all 1,365 test chips.	`POST /api/mstar/pattern`, `GET /api/mstar/pattern/evaluation`
3-Class Segmentation	Segments a 2×2 tile into background, target and shadow. The tab shows how many qubits the current tile needs (3 per pixel) and whether it fits the quantum solver's 2×2 limit.	`POST /api/mstar/shadow`
About page	Reads the workflow, the steps of a quantum run, verification evidence, scaling, robustness, validation and beyond-binary results.	None (static page)

A.2 What happens when a quantum job runs

1. The browser sends the chip, region and QAOA settings. The defaults are 2 layers, up to 15 optimizer iterations and 512 shots per iteration.
2. The server reads the real MSTAR region, builds the MRF energy and solves it exactly by graph cut, which takes milliseconds.
3. It converts the energy to a QUBO and an Ising Hamiltonian; Classiq's cloud synthesizes the QAOA circuit.
4. Each optimizer iteration submits one 512-shot job to Classiq's simulator. Every sampled bitstring is scored on the QUBO energy, and the progress streams back to the interface.
5. A final 2,048-shot sample at the optimized parameters picks the best bitstring seen.
6. The server returns the quantum result beside the classical one, scored against the SARBake mask when one exists, and records the run in the job history.

Each optimizer iteration is therefore its own Classiq job, which is why a run takes about a minute. The platform record in Figure 6 is one such 512-shot job.

A.3 Job history recorded by the application

The history includes the project's own automated tests, sweeps and verification runs as well as manual use of the interface, so it counts repeat runs of the same chip. A.3 and A.4 cover 16 September to 3 November 2026, before the intensity update in section 5, so their energies are not comparable with section 6.

Measure	Binary segmentation	Three-class segmentation
Jobs recorded, 16 September to 3 November 2026	248	87
Of which with QAOA on Classiq's simulator	102	23
Recorded as successful	248	87
QAOA duration, median (range)	66.6 s (26.4 to 319.2 s)	66.8 s (63.5 to 78.8 s)

For the 102 binary QAOA jobs, the table shows how often QAOA reached the exact classical optimum (energy within 0.001) by tile size. They cover 28 different chips.

Tile	Qubits	QAOA jobs	Matched the classical optimum
1×1	1	6	6
2×2	4	2	2
3×3	9	11	11
4×4	16	66	17
5×5	25	17	0

A.4 Jobs run on 3 November 2026

Start (IST)	View	Chip	Tile	Qubits	Classical energy	QAOA energy	Duration (s)
06:13:20	Binary	bmp2_9563_test_1	4×4	16	5.6416	6.0188	81.9
06:21:36	Three-class	bmp2_9563_test_1	2×2	12	0.4259	0.4259	67.3
06:24:28	Binary	slicy_15deg_1	5×5	25	6.178	12.7708	130.9
06:26:24	Binary	bmp2_9563_test_1	4×4	16	5.6416	7.0123	67.0
06:29:03	Binary	bmp2_9563_test_1	4×4	16	5.6416	6.4645	64.4
06:34:22	Three-class	slicy_15deg_1	2×2	12	0.4905	0.4905	63.8

The Classiq platform record in Figure 6 (completed 6:22 AM, 512 shots, 12-bit outcomes, job ID 8144bd99-e67e-401b-815e-164b2a398987) corresponds to the 06:21:36 three-class run on BMP2 #9563, by completion time and qubit count. The three runs of the same 4×4 tile returned QAOA energies of 6.0188, 7.0123 and 6.4645 against one classical optimum of 5.6416, which shows run-to-run variation in QAOA.

After the intensity update (4 November 2026). The section 6 sweeps ran 12 QAOA jobs on Classiq's simulator (six tile sizes, six noise levels), plus the run in Figures 2 and 3. All succeeded, in 60.0 to 127.5 s.

Bibliographic references

Numbers match the citations in the text. References 17 to 20 are those listed in the THALES brief [16].

1. D. M. Greig, B. T. Porteous and A. H. Seheult (1989). Exact maximum a posteriori estimation for binary images. *Journal of the Royal Statistical Society: Series B* 51(2), 271–279. doi:10.1111/j.2517-6161.1989.tb01764.x
2. Y. Boykov and M.-P. Jolly (2001). Interactive graph cuts for optimal boundary and region segmentation of objects in N-D images. *Proc. IEEE ICCV*, vol. I, 105–112. doi:10.1109/ICCV.2001.937505
3. Y. Boykov, O. Veksler and R. Zabih (2001). Fast approximate energy minimization via graph cuts. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 23(11), 1222–1239. doi:10.1109/34.969114
4. V. Kolmogorov and R. Zabih (2004). What energy functions can be minimized via graph cuts? *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26(2), 147–159. doi:10.1109/TPAMI.2004.1262177
5. A. Lucas (2014). Ising formulations of many NP problems. *Frontiers in Physics* 2, 5. doi:10.3389/fphy.2014.00005
6. E. Farhi, J. Goldstone and S. Gutmann (2014). A quantum approximate optimization algorithm. arXiv:1411.4028. arxiv.org/abs/1411.4028
7. S. H. Sack and M. Serbyn (2021). Quantum annealing initialization of the quantum approximate optimization algorithm. *Quantum* 5, 491. doi:10.22331/q-2021-07-01-491
8. M. J. D. Powell (1994). A direct search optimization method that models the objective and constraint functions by linear interpolation. In S. Gomez and J.-P. Hennart (eds), *Advances in Optimization and Numerical Analysis*, Kluwer Academic, 51–67. (The COBYLA method.)
9. S. Otgonbaatar and M. Datcu (2020). Quantum annealing approach: feature extraction and segmentation of synthetic aperture radar image. *Proc. IEEE IGARSS 2020*, 3692–3695. doi:10.1109/IGARSS39084.2020.9323504
10. E. R. Keydel, S. W. Lee and J. T. Moore (1996). MSTAR extended operating conditions: a tutorial. *Proc. SPIE* 2757, 228–242. doi:10.1117/12.242059
11. T. D. Ross, S. W. Worrell, V. J. Velten, J. C. Mossing and M. L. Bryant (1998). Standard SAR ATR evaluation experiments using the MSTAR public release data set. *Proc. SPIE* 3370, 566–573. doi:10.1117/12.321859
12. A. Mahalanobis, B. V. K. Vijaya Kumar, S. Song, S. R. F. Sims and J. F. Epperson (1994). Unconstrained correlation filters. *Applied Optics* 33(17), 3751–3759. doi:10.1364/AO.33.003751
13. B. Bhanu and G. Jones III (2000). Recognizing target variants and articulations in synthetic aperture radar images. *Optical Engineering* 39(3), 712–723. doi:10.1117/1.602418
14. Classiq Technologies. Classiq SDK and Qmod open library (`qaoa_init`, `qaoa_layer`). classiq.io
15. U.S. Air Force Research Laboratory. MSTAR Public Targets Chips (T72, BMP2, BTR70, SLICY), Sensor Data Management System. sdms.afrl.af.mil
16. THALES. *THALES Use-Case: SAR (Synthetic Aperture Radar) Image Segmentation*. Use-case brief supplied to participants (4 pages).
17. S. Huang, W. Huang and T. Zhang (2016). A new SAR image segmentation algorithm for the detection of target and shadow regions. *Scientific Reports* 6, 38596. doi:10.1038/srep38596
18. R. Shang, J. Lin, L. Jiao and Y. Li (2020). SAR image segmentation using region smoothing and label correction. *Remote Sensing* 12(5), 803. doi:10.3390/rs12050803
19. F. Galland, J.-M. Nicolas, H. Sportouche, M. Roche, F. Tupin et al. (2009). Unsupervised synthetic aperture radar image segmentation using Fisher distributions. *IEEE Transactions on Geoscience and Remote Sensing* 47(8), 2966–2972. HAL: hal-00437471.
20. H. Kunsch, S. Geman and A. Kehagias (1995). Hidden Markov random fields. *Annals of Applied Probability* 5(3), 577–602. doi:10.1214/aoap/1177004696
21. D. Malmgren-Hansen and M. Nobel-Jørgensen (2015). Convolutional neural networks for SAR image segmentation. *Proc. IEEE ISSPIT 2015*. Masks: *SARBake Overlays for the MSTAR Dataset*, Mendeley Data, v3 (2017), CC BY 4.0. doi:10.17632/jxhsg8tj7g.3